

Attack Surface Report — EH-A1 Recon Engine and Foothold

Stage: 5 · **Target:** local loopback lab (127.0.0.1) · **Engine:** Python stdlib only

1. Scope

Two loopback endpoints were authorized by the generated scope: an HTTP service and a line-protocol "signal" service. A third loopback port was marked OUT as a scope-discipline decoy, and all non-loopback destinations were prohibited. The engine enforces scope before every socket is opened; the decoy received zero packets (see request-ledger.csv).

2. Methodology

Discovery proceeded in scope-safe stages, each routed through a single gate that checks authorization and records every request to a ledger:

1. Port discovery: connect-scan of the authorized ports only.
2. Signal service: read the banner, issue CAPS to learn the advertised commands, then ROUTE to obtain the virtual host and route key. The service answers one command per connection, so a fresh connection is used per command.
3. HTTP / virtual host: a wildcard baseline was established using a random, non-existent hostname. The genuine virtual host produced a response that differed from the baseline (retained), confirming real content rather than catch-all noise.
4. Credential discovery: a diagnostics endpoint behind the genuine vhost disclosed service credentials.

3. Findings

- The HTTP service serves a stable wildcard page to unknown hostnames, hiding its real content behind a specific virtual host.
- The signal service discloses the virtual host and a route-proof key.
- A diagnostics path behind the real vhost discloses a username and password.
- Reaching the protected resource requires HTTP Basic authentication and a route-key header, i.e. proof from both services, a deliberate multi-service chain.

4. Foothold

Using only values discovered by the engine (virtual host, route key, username, password), a single authenticated request to /user.txt returned the runtime flag. The full request/response transcript is in foothold-evidence.txt, and the request reconciles with the lab's own request ledger. The recon engine itself performs discovery only and never fetches the flag.

5. Scope and safety

Every request is logged before it is sent. The engine's ledger and the lab's independent ledger agree, and neither shows any contact with the decoy or any non-loopback destination. The full run used well under the 240-request budget.

6. Reproducibility

The engine reproduces from a clean environment (make test, make run) and produces a deterministic result hash that is identical across clean and interrupted or resumed runs. See assessment-manifest.json for exact commands and the expected hash.